Agente coordinador de búsqueda y rescate andino:
planificación adaptativa con LangGraph, memoria y RAG

Evaluación Parcial 2 · ISY0101 Ingeniería de Soluciones con IA
Benjamín Ibáñez · Sección: 001D · Docente: Giocrisrai Godoy Bonillo
19 de octubre de 2026
Repositorio: https://github.com/beibanezv/ep2-rescate-agente

Resumen

Este informe presenta un agente que convierte el reporte de una persona desaparecida en un plan de búsqueda por fases y lo replanifica cuando el clima invalida la ventana de un medio. La decisión se calcula con código determinista y seis tools de dominio; un grafo LangGraph de cinco nodos orquesta contexto, cálculo, publicación, redacción y memoria. El modelo de lenguaje solo redacta el parte. El agente conserva memoria de sesión, recupera protocolo y criterios por búsqueda semántica y publica planes validados. Se verificó con 14 pruebas y dos actos end-to-end ejecutados con Groq real: el plan v1 asigna helicóptero y dron dentro de su ventana; el plan v2 descarta HEL-1 en ambas zonas y DR-1 en Z-2, conserva DR-1 en Z-5 y refuerza el barrido terrestre.

1. Introducción y contexto

La coordinación de un operativo de búsqueda y rescate (SAR) exige priorizar zonas, asignar medios y justificar cada decisión ante un protocolo. En un caso real, el pronóstico cambia y obliga a replanificar. El objetivo de este trabajo es construir un agente que reciba el reporte, consulte los datos del caso y produzca un plan trazable, con memoria entre turnos y capacidad de adaptarse a un evento de viento (ISY0101, 2026).

El caso simulado ocurre en la noche del 17 al 18 de octubre de 2026 en El Morado. La ficha indica último contacto en el mirador (Z-2), ruta declarada y una persona sin equipo técnico. El corpus fija prioridades y reglas de viabilidad. La contribución es un flujo que separa decisión (Python) de narración (LLM), con herramientas de lectura y una tool de escritura que valida antes de persistir.

2. Arquitectura y orquestación

El flujo se implementa en agent/grafo.py con LangGraph (LangChain, 2026). El estado EstadoPlan es un TypedDict que viaja por cinco nodos: contexto, calcular, publicar, redactar y memorizar (Figura 1). contexto recupera el plan previo del buffer y los recuerdos semánticos de Chroma, arma los hechos de tools ([T#]) y busca fragmentos del corpus ([F#]). calcular aplica la fórmula de prioridad y construye las fases. publicar llama a publicar_plan, que valida y escribe data/planes/plan-vN.json. redactar pide al LLM el parte narrativo con el plan ya publicado. memorizar registra el turno en el buffer y en Chroma.

La orquestación con LangGraph se eligió frente a CrewAI porque el problema es un pipeline con estado explícito y control determinista, no una conversación entre roles. LangGraph permite inspeccionar el estado, imponer el orden y trazar cada nodo; una tripulación de agentes habría añadido variabilidad y costo sin mejorar la decisión. El grafo se compila una vez en AgenteRastreo.__init__ y planificar() lo invoca, de modo que el CLI y las pruebas usan el mismo camino.

El LLM no decide: recibe el plan validado y redacta. En producción se usa ClienteLangChain con Groq openai/gpt-oss-120b (Groq, 2026); en pruebas se usa ClienteFalso, determinista y sin cuota. Los actos 1 y 2 de docs/ se ejecutaron con Groq real. LangSmith es opcional y se activa solo si hay LANGSMITH_API_KEY.

2.1 Separación entre decisión y narración

La separación responde a un requisito de auditoría: cada fase debe poder reconstruirse desde los JSON y el protocolo, sin depender del muestreo del LLM. Por eso calcular y publicar son funciones deterministas sobre datos versionados; redactar recibe el plan ya escrito y no puede alterarlo. Esta frontera también abarata las pruebas: el grafo se ejecuta con ClienteFalso y produce el mismo plan que con Groq, porque el plan no pasa por el modelo.

El estado EstadoPlan hace explícito el contrato entre nodos: version, sesion_id, consulta, recuerdo, herramientas, fragmentos, fases, prioridad, motivo, citas, plan, archivo, parte, fuentes, cito_plan_previo y archivo_trace. Cada nodo lee lo anterior y agrega su salida; LangGraph conserva el esquema y la trazabilidad.

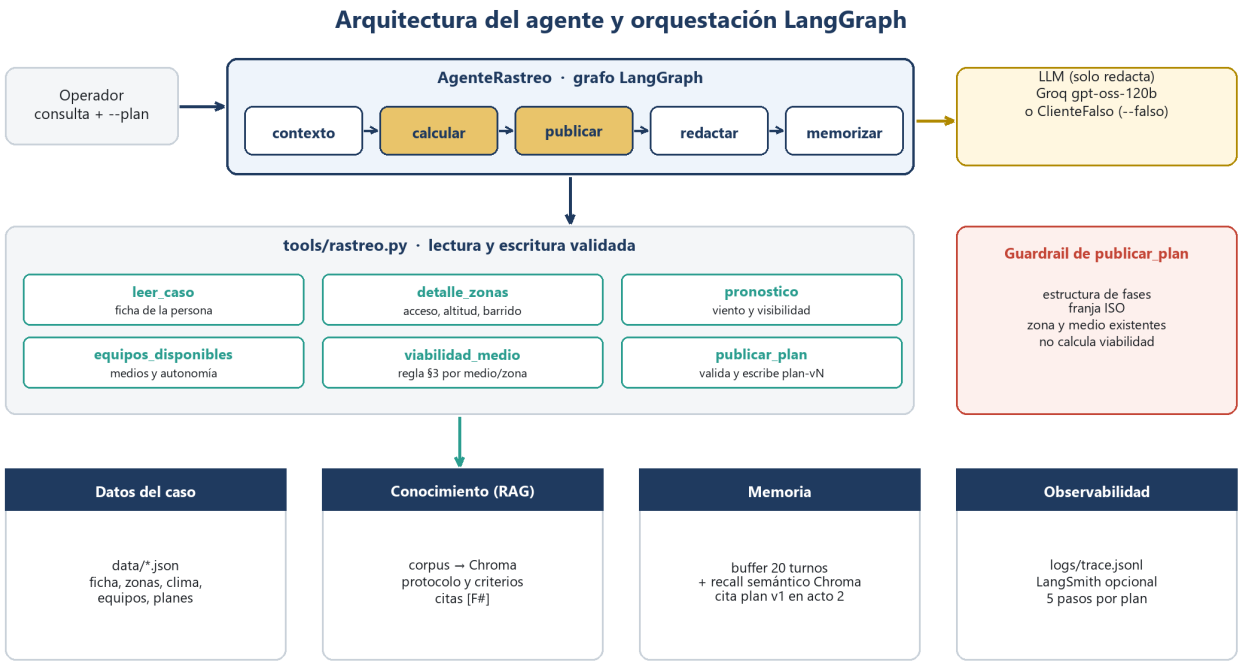


Figura 1. Arquitectura del agente. El grafo LangGraph orquesta cinco nodos; las tools leen los JSON y el corpus, y publicar_plan valida antes de escribir.

3. Tools, autonomía y guardrails

El agente dispone de seis tools (Tabla 1). Las de lectura exponen el caso, las zonas, el pronóstico y los equipos. viabilidad_medio codifica el protocolo §3: HEL-1 requiere viento < 35 km/h y visibilidad > 5 km; DR-1 tolera hasta 40 km/h; ambos solo operan de día (06:00–18:00); las brigadas y K9 no ingresan sobre 3000 m sin aclimatación. publicar_plan es la única tool de escritura: valida versión, fases, franjas ISO, zonas y medios contra los JSON, y rechaza cualquier estructura inválida antes de escribir.

Tool	Tipo	Qué resuelve
leer_caso	lectura	ficha, último contacto y ruta declarada
detalle_zonas	lectura	acceso, altitud, barrido y refugios
pronostico	lectura	viento, visibilidad y temperatura por franja
equipos_disponibles	lectura	medios, personas, autonomía y restricciones
viabilidad_medio	cálculo	aplica la regla §3 a medio, zona y franja
publicar_plan	escritura	valida y persiste data/planes/plan-vN.json

Tabla 1. Tools de dominio y su función.

El flujo es un pipeline determinista: el nodo contexto ejecuta siempre la recuperación y las verificaciones de dominio, y la plantilla de fases es fija. La autonomía está en el cálculo de prioridades y en la selección de medios viables, no en una elección libre de tools por el LLM. La secuencia queda en logs/trace.jsonl (planificación, memoria semántica, plan publicado, parte, memoria). El guardrail central es que el plan se calcula en Python y el LLM no puede introducir zonas o medios: aunque alucine, el texto no se convierte en plan; solo se escribe lo que pasó por publicar_plan (Russell & Norvig, 2021).

4. Memoria y recuperación semántica

La memoria de corto plazo es un buffer JSON por sesión (memoria/sesiones.json) que conserva los últimos 20 turnos con consulta, resumen y campos de dominio (plan_version, plan_archivo, plan_resumen). contexto_para_prompt inyecta los últimos turnos al prompt. La memoria de largo plazo es una colección Chroma memoria con embeddings locales paraphrase-multilingual-MiniLM-L12-v2 (Reimers & Gurevych, 2019), filtrada por sesion_id; recuperar devuelve recuerdos con fuente y distancia (Chroma, 2026).

El RAG usa una colección conocimiento con el protocolo y los criterios de prioridad; el parte narrativo cita fragmentos [F#] y hechos de tools [T#], mientras el JSON del plan guarda citas legibles. En el acto 2, cito_plan_previo=True y las fuentes incluyen memoria-semantica:caso-morado y memoria:caso-morado; el parte recibe el plan v1, y el motivo del cambio se escribe en el plan v2. La recuperación semántica no se limita a repetir el buffer: Chroma permite buscar por significado y filtrar por sesión, lo que sostiene la continuidad entre turnos y la justificación histórica (Lewis et al., 2020).

5. Planificación priorizada y adaptación

La prioridad aplica la fórmula del corpus: score = probabilidad × facilidad / barrido_terrestre_h. La probabilidad es 1,0 si la zona contiene el último contacto, 0,75 si está en la ruta declarada, 0,5 si tiene refugios y 0,25 en el resto. La facilidad suma 0,2 si el dron es adecuado y 0,2 si el acceso es vehicular. El resultado en el caso es Z-2 > Z-5 > Z-1 > Z-4 > Z-3. La fase nocturna ancla las dos zonas probables más fáciles (Z-2 y Z-5) con EQ-2 y K9-1, sin zonas técnicas y antes del amanecer. La fase diurna barre todas las zonas probables; asigna HEL-1 y DR-1 solo si viabilidad_medio lo permite, y EQ-1 cuando la zona queda fuera de la ruta o no hay medio aéreo.

El acto 1 publica el plan v1 con HEL-1 y DR-1 en Z-2 y Z-5 entre 06:00 y 12:00. El acto 2 reemplaza esas franjas por viento de 41 km/h en Z-2 y 38 km/h en Z-5. viabilidad_medio descarta HEL-1 en ambas por la regla de 35 km/h y DR-1 en Z-2 por superar 40 km/h; DR-1 se mantiene en Z-5. El plan v2 refuerza Z-2 con EQ-1, conserva DR-1 en Z-5, mantiene EQ-1 en Z-1 y la fase nocturna. El motivo se escribe con la regla citada y se propone reevaluar HEL-1 en 12:00–18:00, cuando el pronóstico baja a 24, 22 y 26 km/h (Figura 2). La reevaluación sigue el ciclo de razonar, actuar y volver a observar el entorno de ReAct (Yao et al., 2023).

Zona	Plan v1 (06:00–12:00)	Plan v2 (06:00–12:00)	Motivo del cambio
Z-2	HEL-1 + DR-1	EQ-1	viento 41 km/h: HEL-1 >= 35 y DR-1 > 40
Z-5	HEL-1 + DR-1	DR-1	viento 38 km/h: HEL-1 >= 35; DR-1 <= 40
Z-1	EQ-1	EQ-1	sin cambio de medio
Noche	EQ-2 + K9-1 (Z-2, Z-5)	sin cambios	la ventana nocturna sigue vigente

Tabla 2. Comparación de los planes v1 y v2 para la ventana diurna.

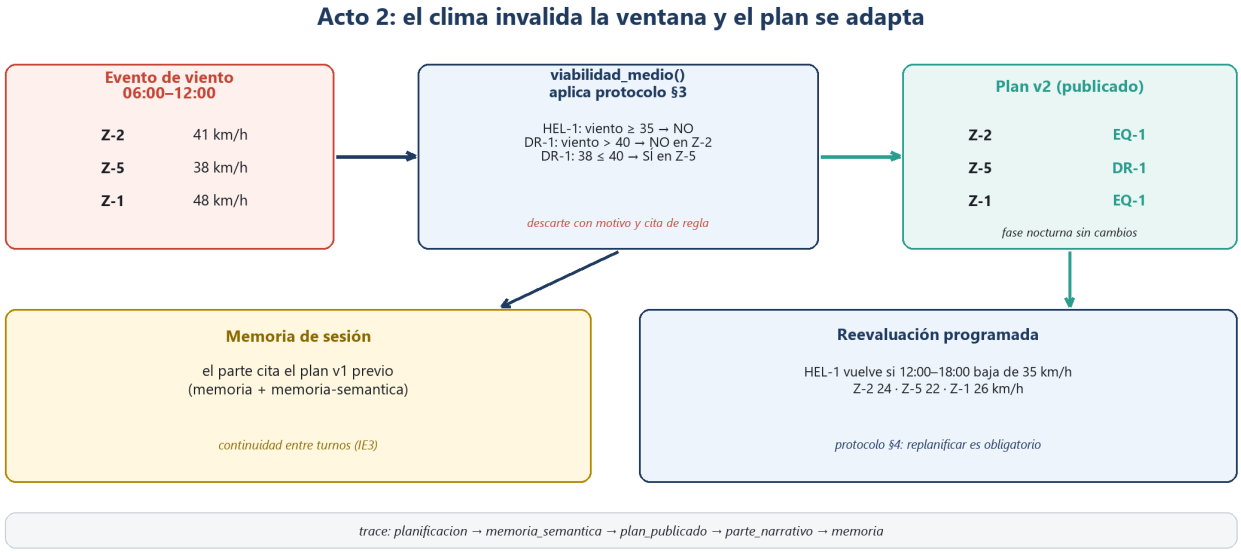


Figura 2. Flujo de adaptación del acto 2: el evento de viento se evalúa con la regla, se descarta el medio inviable y se publica un plan nuevo con reevaluación programada.

6. Verificación y evidencias

La suite tiene 14 pruebas: 8 del motor (chunking, cliente falso, memoria, agente, puerta de dominio, tool de escritura, cliente LangChain y citas con espacios) y 6 del dominio (plan v1, evento de viento, seguridad nocturna, grafo completo, validación de publicar_plan y recuperación semántica en contexto). Todas pasan con uv run python -m pytest -q (14/14). La ingesta carga 2 documentos y genera 7 fragmentos; las dos consultas de verificación devuelven MATCH (2/2): reglas de viento y fórmula de prioridad.

Los actos se ejecutaron end-to-end con Groq real (ClienteLangChain + gpt-oss-120b) y quedaron en docs/salida-acto1.txt y docs/salida-acto2.txt, con rutas relativas y la traza de cinco pasos. El plan v2 se conserva en docs/plan-v2-evidencia.json. La Tabla 3 mapea cada indicador de la pauta con su evidencia.

IE	Evidencia en el entregable	Archivo
IE1	Seis tools de consulta/cálculo/escritura y pipeline consulta→contexto→plan→parte; el LLM solo redacta	tools/rastreo.py; agent/agente_base.py
IE2	Grafo LangGraph de 5 nodos con estado tipado; planificar() delega en el grafo	agent/grafo.py; agent/rastreo.py
IE3	Buffer de 20 turnos; el acto 2 cita el plan v1 de la sesión	agent/memoria.py; docs/salida-acto2.txt
IE4	Buffer + Chroma; `recuperar` por sesión en el nodo contexto y recuperación semántica del corpus	agent/memoria.py; agent/retriever.py
IE5	Fórmula probabilidad × facilidad / barrido y fases nocturna/diurna priorizadas	agent/rastreo.py
IE6	Viento 41/38 km/h → descarte de HEL-1/DR-1 y plan v2 con reevaluación	tests/test_rastreo.py; docs/salida-acto2.txt
IE7	Figura 1, diagrama Mermaid y README con ejecución y arquitectura	README.md; este informe
IE8	Justificación de LangGraph, tools deterministas, memoria y embeddings locales	Secciones 2–5
IE9	Informe de 4 páginas con dos figuras, tres tablas y referencias APA	docs/informe-ep2.docx; docs/informe-ep2.pdf
IE10	Lenguaje técnico, transcripciones, pruebas y evidencia reproducible en el repositorio	docs/salida-acto*.txt; tests/

Tabla 3. Correspondencia entre los indicadores de la EP2 y la evidencia del entregable.

7. Limitaciones y conclusión

El caso es simulado y único; el corpus es pequeño y el parte narrativo no se evaluó con métricas de calidad. La memoria y el RAG dependen de Chroma y de la descarga local de embeddings. El agente no reemplaza al coordinador: propone un plan validado y explicable. Como trabajo futuro se propone ampliar el corpus, medir la calidad del parte con una rúbrica y validar el plan con datos operativos reales.

El resultado cubre los indicadores de tools, framework, memoria, recuperación, planificación y adaptación, y deja el informe y la evidencia reproducibles en el repositorio. La separación entre decisión determinista y narración LLM es la decisión de diseño que hace el sistema auditable.

Referencias

Chroma. (2026). Chroma documentation. https://docs.trychroma.com/

Groq. (2026). Groq API documentation. https://console.groq.com/docs

ISY0101. (2026). Evaluación Parcial 2: Instrucciones [Documento de curso]. Ingeniería de Soluciones con IA.

LangChain. (2026). LangGraph documentation. https://langchain-ai.github.io/langgraph/

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Riedel, S. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474. https://arxiv.org/abs/2005.11401

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of EMNLP-IJCNLP 2019, 3982–3992. https://arxiv.org/abs/1908.10084

Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations. https://arxiv.org/abs/2210.03629